

Reviewer Pack — Secant Rank Lower Bound for Disjointness Communication Matri...

Entry 902c529a20f5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 12:15:03 UTC

Secant Rank Lower Bound for Disjointness Communication Matrices

Entry ID: 902c529a20f5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 12:15:03 UTC

1. Conjecture

Field A (mathematical branch): Algebraic geometry of secant varieties **Field B** (complexity object): Communication complexity of DISJOINTNESS

Statement:

For the $n \times n$ DISJOINTNESS matrix M_n , the secant rank $\text{sr}(M_n)$ satisfies $\text{sr}(M_n) \geq \Omega(n)$. This bound is tight for all $n \geq 1$, with equality achieved when M_n is the incidence matrix of a projective plane of order n .

Rationale (proposer's reasoning):

Secant rank measures the minimal number of rank-1 tensors needed to span a matrix, reflecting its geometric complexity. For DISJOINTNESS, this invariant exposes the inherent structure of disjointness constraints, which are algebraically rigid due to their incidence geometry. The projective plane construction provides a tight example where the secant rank scales linearly with n .

Taxonomy category: COMM_DISJ (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c84de56c08e07e82

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_disjointness_matrix(n):
        M = [[0] * n for _ in range(n)]
        for i in range(n):
            M[i][i] = 1
        return M

    def secant_rank(M):
        n = len(M)
        minors = []

        def determinant(matrix):
            if len(matrix) == 2:
                return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
            det = 0
            for j in range(len(matrix)):
                submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
                det += (-1) ** j * matrix[0][j] * determinant(submatrix)
            return det

        def is_rank_1(M):
            if len(M) == 1 and len(M[0]) == 1:
                return True
            for i in range(len(M)):
                for j in range(len(M[i])):
                    if M[i][j] != 0:
                        submatrix = [row[:j] + row[j+1:] for row in M[:i] + M[i+1:]]
                        if determinant(submatrix) == 0:
                            return True
            return False

        def compute_minors(M):
```

```

    n = len(M)
    for i in range(n):
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in M[:i] + M[i+1:]]
            minors.append(determinant(submatrix))

    compute_minors(M)
    rank_1_tensors = 0
    for minor in minors:
        if is_rank_1([[minor]]):
            rank_1_tensors += 1

    return rank_1_tensors

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    M = generate_disjointness_matrix(n)
    sr_M_n = secant_rank(M)
    results.append(sr_M_n)

    if sr_M_n < 0.8 * n:
        return {
            "metric_name": "secant_rank",
            "metric_value": sr_M_n,
            "instances_tested": len(results),
            "conjecture_holds": False,
            "counterexample": f"n={n}, sr(M_n)={sr_M_n}"
        }

support_fraction = sum(1 for r in results if r >= 0.8 * n_values[-1]) / len(results)
return {
    "metric_name": "secant_rank",
    "metric_value": sum(results) / len(results),
    "instances_tested": len(results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30*100 + 1, 100))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["support_fraction"])

    mean_support_fraction = sum(results) / len(results)
    support_count = sum(1 for r in results if r >= 0.8)
    support_fraction = support_count / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_support_fraction} std=NA support_fraction={support_fr")

```

```

elif any(r < 0.8 for r in results):
    first_failing_seed = seeds[results.index(min(results))]
    print(f"RESULT: FALSIFIED counterexample=\"n=5, sr(M_n)=0\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing results; cannot evaluate support fraction or counterexamples. | next: Increase timeout duration and re-run the test with identical parameters

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	50362
2	propose	ollama_remote	qwen3:8b	0	0	37720
3	propose	ollama_remote	qwen3:8b	0	0	68644
4	preregistration	ollama_remote	qwen3:8b	0	0	27619
5	novelty	ollama_remote	qwen3:8b	0	0	24375
6	novelty	ollama_remote	qwen3:8b	0	0	20266
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17481
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9763
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8665
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10746
11	judge	ollama_remote	qwen3:8b	0	0	18613

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 294255 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in [research/audit/902c529a20f5.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/902c529a20f5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/902c529a20f5.tar.gz` (if generated)